

FRONT END DEVELOPMENT

WEEK 9 – Going deeper with States





What will you learn today?



- ✓ Reflect on a **meaningful usage of states**
- ✓ Understand React.JS lifecycle and **asynchronous** execution
- ✓ Understand the benefits of a **Virtual DOM**
- ✓ Passing **functions as value** to props



10 MIN

Activity 1

- ✓ Connect to the sandbox
- ✓ Your task is to **dynamically apply a CSS class** (active) to the `<p>Style me</p>` element
- ✓ The class should be applied when the `<button>` is clicked, then removed, then applied gain etc.
- ✓ Before coding : complete this table , to explain **the usage of your states**:

State Name	Type	Role	Used for...	Changed when...



Activity 1

State Name	Type	Role	Used to...	Changed when...
selected	Boolean	Store whether button is selected or not	Decide the text class	Click on button

```
const [selected, setSelected] = React.useState(false);

function handleToggle() {
  | setSelected(!selected);
}

function getTextClass() {
  | return selected ? "active" : "";
}

return (
  <div>
  |   <p className={getTextClass()}>Style me!</p>
  |   <button onClick={handleToggle}>Toggle style</button>
  </div>
);
```



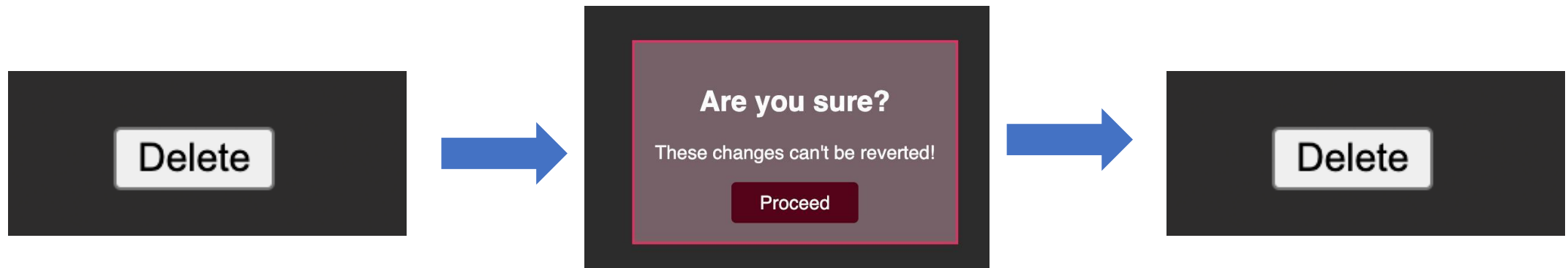
10 MIN

Activity 2

Conditional Displays

- ✓ Connect to the sandbox
- ✓ Conditionally show a warning dialog once a user has clicked **DELETE** button.
- ✓ Inside that dialog, dismiss the warning (*i.e., remove the warning box from the screen*) once a user has clicked **PROCEED** button
- ✓ Before coding : complete this table , to explain **the usage of your states**:

State Name	Type	Role	Used for...	Changed when...



Activity 2

Conditional Displays

State Name	Type	Role	Used to...	Changed when...
dialogVisible	Boolean	Manage the visibility of the dialog	Hide/show dialog Hide/show delete button	Click on PROCEED Click on DELETE

&& syntax for
conditional displays

```
const [dialogVisible, setDialogVisible] = React.useState(false);

return (
  <div>
    {dialogVisible && (
      <div id="alert">
        <h2>Are you sure?</h2>
        <p>These changes can't be reverted!</p>
        <button onClick={(e) => setDialogVisible(false)}>Proceed</button>
      </div>
    )}
    {!dialogVisible && (
      <button onClick={(e) => setDialogVisible(true)}>Delete</button>
    )}
  </div>
)
```



10 MIN

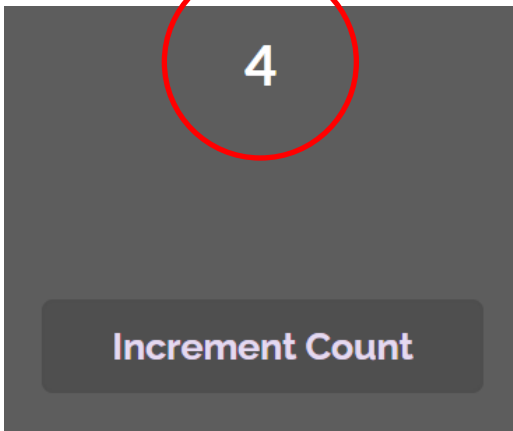
Activity 3

- ✓ **Unzip, install and run** the app
- ✓ In this app, we increment a **stateCount** and a **localCount** upon button click
- ✓ Look at the console logs, and answer the 2 questions :

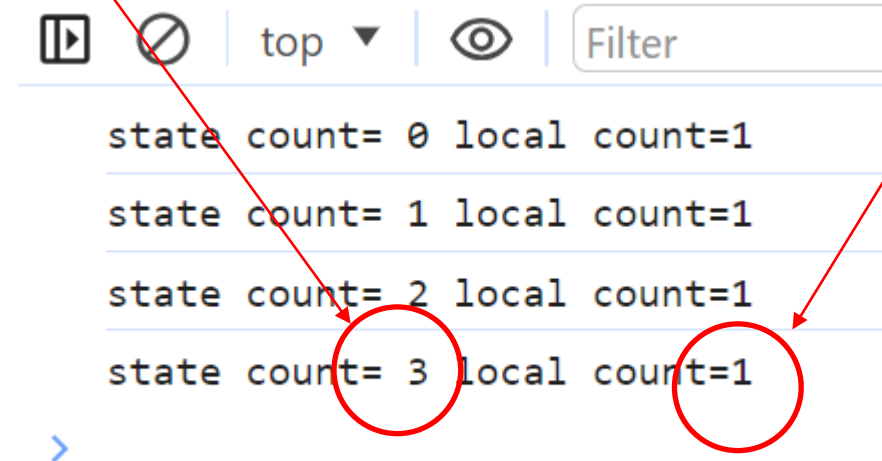
Q1: Why the local count stay at 1?

Q2: Why the state count is late of 1 compared to the view?

The view display 4



The state count is still at 3

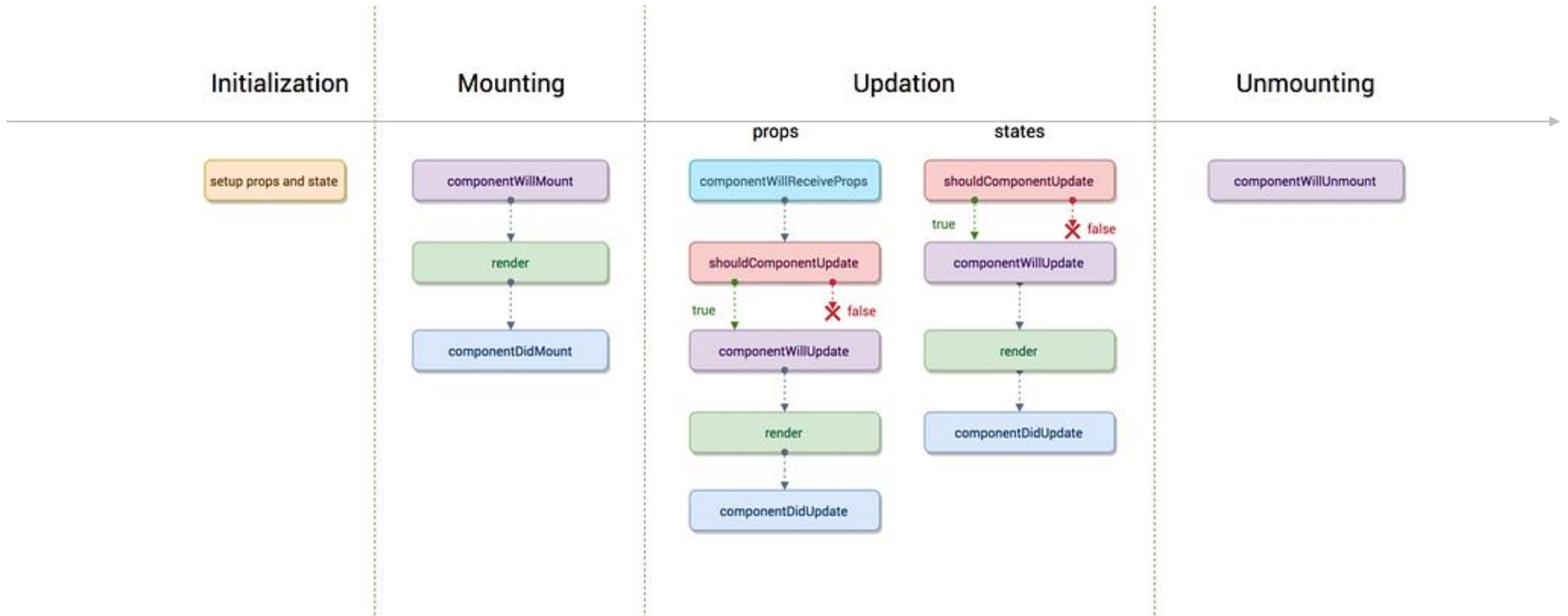


The local count does not change

Q1: Why the local count stays at 1?

[READ MORE HERE](#)

Because of the React component lifecycle !



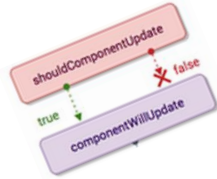
What are the 3 cases when React will **render** a component ?

Q1: Why the local count stays at 1?

Because of the React component lifecycle !

1

We render component
Whenever its **state changes**



2

Local variables don't persist
between renders

3

State variables persist
between renders

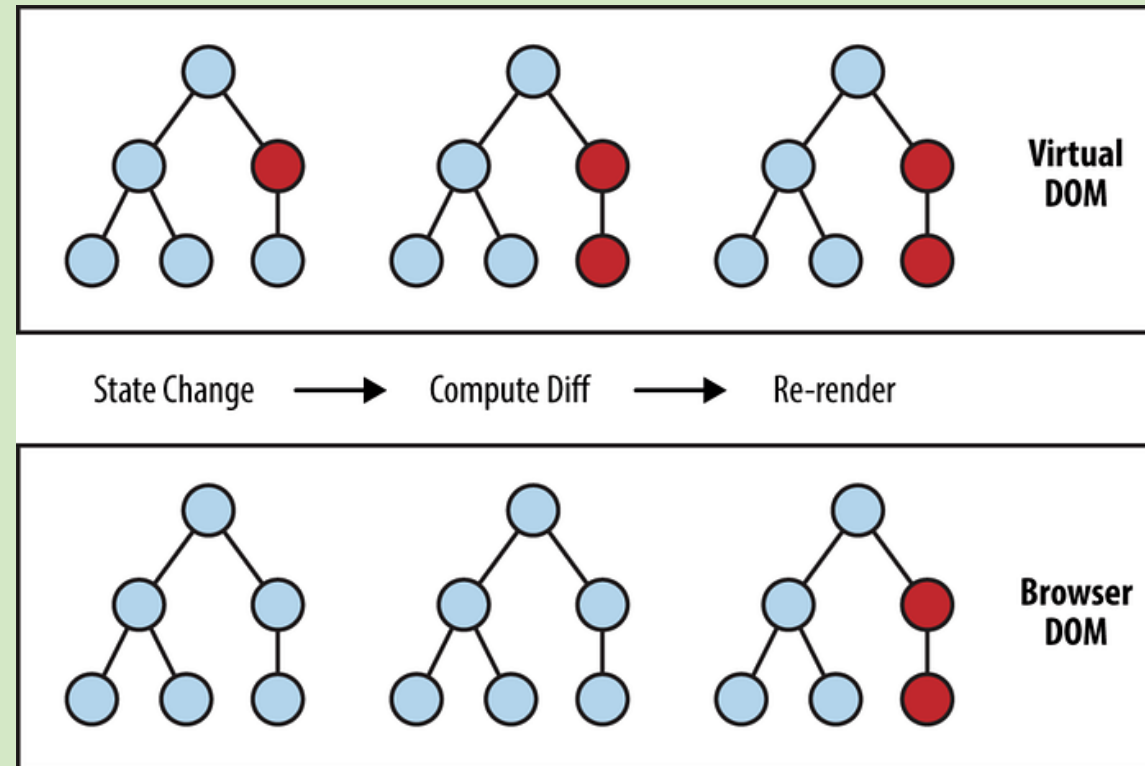
```
function App() {  
  const [stateCount, setStateCount] = React.useState(0);  
  let localCount = 0;  
  
  const incrementCount = () => {  
    setStateCount(stateCount + 1);  
    localCount++;  
  
    console.log("state count= " + stateCount + " local count=" + localCount);  
  };  
  
  return (  
    <>  
      <main>  
        <h1>{stateCount}</h1>  
        <button onClick={incrementCount}>Increment Count</button>  
      </main>  
    </>  
  );  
}
```



Virtual DOM VS Browser DOM

[LEARN MORE HERE](#)

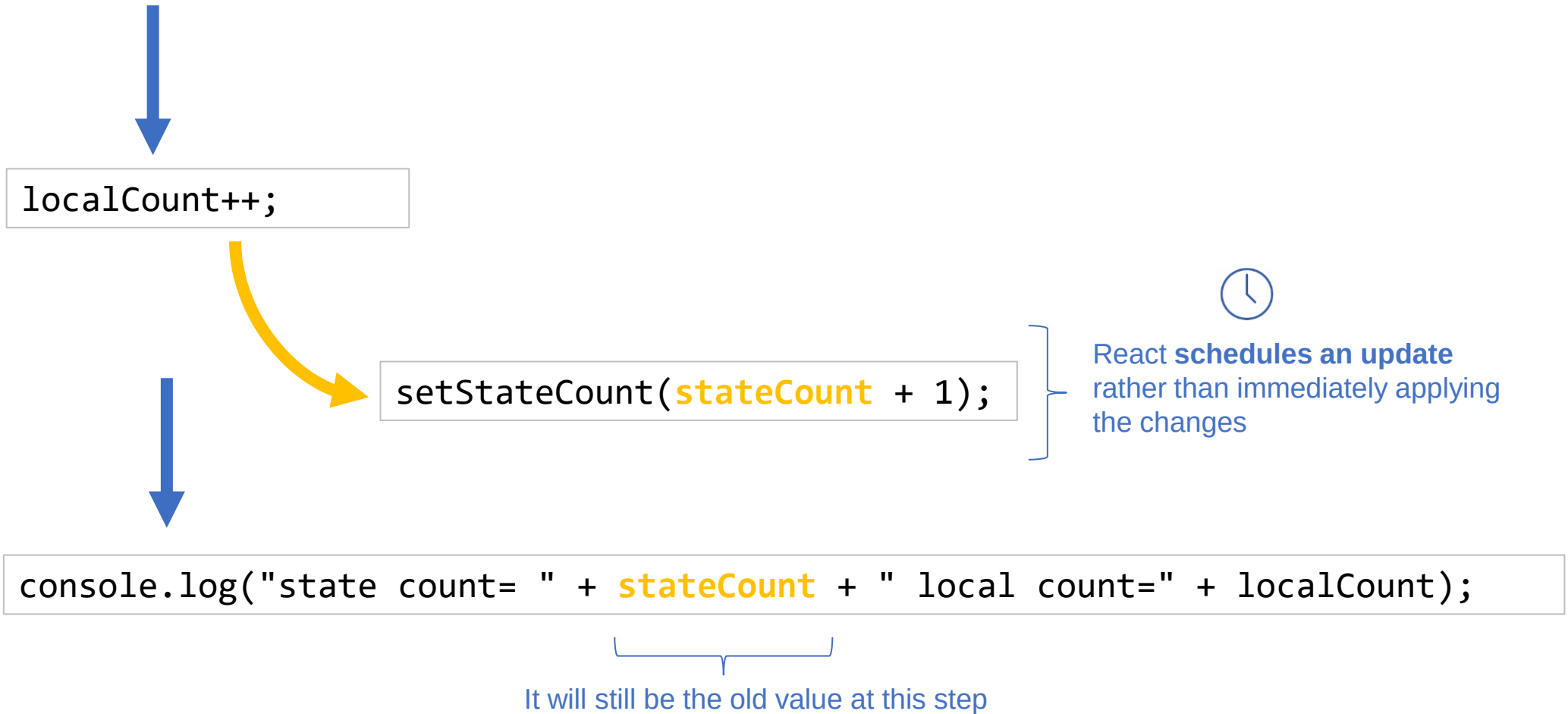
- ✓ Manipulating the **real browser** DOM is **slow**
- ✓ Manipulating the **virtual DOM** is much faster because nothing gets drawn on screen.
- ✓ By working on a virtual DOM, React ensures that there are **minimal operations** on the real DOM.



React JS works on a Virtual representation of the DOM

Q2: Why the state count is late ?

Because state updates are done asynchronously !



Updating state based on the previous state

Use an asynchronous function !

USING THE NORMAL SETTER

```
setCount(count +1);  
setCount(count +1);
```

This will not work as
the state update is
asynchronous !

USING AN UPDATER FUNCTION

```
setCount(prevCount => prevCount +1);  
setCount(prevCount => prevCount +1);
```

The previous state

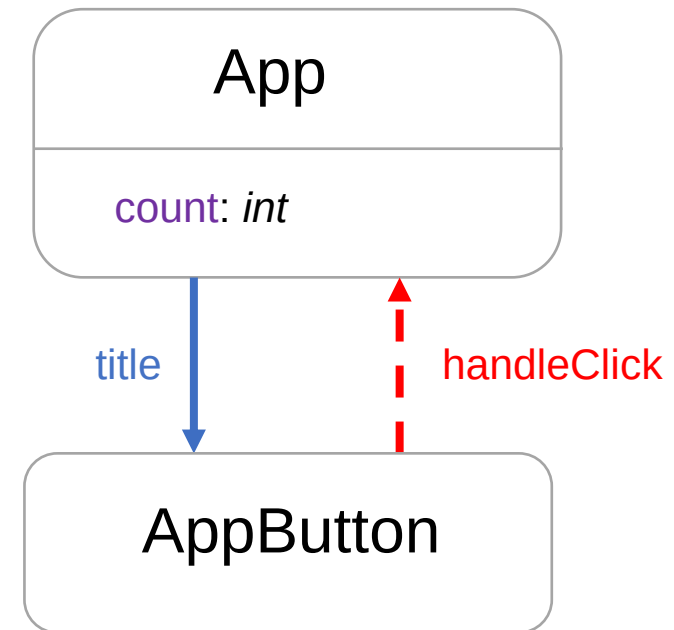
Your code to
update the state

Passing functions as value to props

[LEARN MORE HERE](#)

- ✓ Passing a callback function to props will **trigger actions in the parent component**
- ✓ *You can need it as example to update the state of the parent*

```
function AppButton({ title, handleClick }) {  
  return <button onClick={handleClick}>{title}</button>;  
}  
  
export default function App() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    console.log("Function ran in Child component");  
    setCount(count + 1);  
  }  
  
  return (  
    <div>  
      <h2>Count is: {count}</h2>  
  
      <Child title="Add" handleClick={handleClick} />  
    </div>  
  );  
}
```



Component diagram representing the interactions

Passing VS Calling

*Functions passed to event handlers **must be passed, not called.***

PASSING

```
<button onClick={ handleClick }>
```

YES!

- ✓ The function is **passed** as an on ClickEvent handler.
- ✓ React will be remember it and only **call the function when the user clicks** the button.

CALLING

```
<button onClick={ handleClick() }>
```

NO!

- ✓ The function will be executing immediately **during rendering**, without any clicks.
- ✓ Because **JavaScript inside the JSX { and } executes right away.**



10 MIN

Activity 4

- ✓ Connect to the sandbox
- ✓ We need to **select the BEST JOKE** - *only 1 best joke!*
- ✓ The best joke has a **blue background** color (*selected class*)
- ✓ Before coding :
 - Reflect on the **states** you need
 - Reflect on **which component** should hold your states
- ✓ After coding : draw the component diagram of your app

Component	State Name	Type	Role	Used for...	Changed when...

Vietnamese Jokes

Why was the math book sad? Because it had too many problems

Likes 2



Why was the computer cold? Because it left its Windows open

Likes 2



Why did the banana go to the doctor? Because it wasn't peeling well!

Likes 2

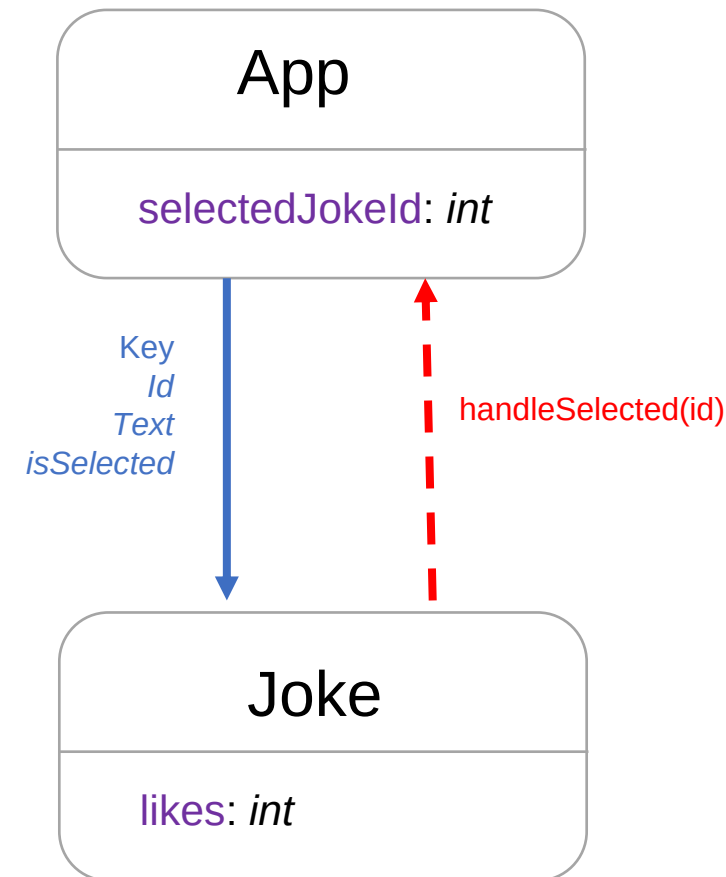


The selected best joke !

Activity 4

- ✓ We use **the joke ID** to store which joke is selected
- ✓ This ID must be in **managed in the parent component** (App)
- ✓ Therefore, we **need a callback** to handle a selection in the App component

Component	State Name	Type	Role	Changed when...
Joke	likes	int	Manage the number of like	Click on like/dislike
App	selectedJokeID	int	Manage the selected best joke	Click on a joke





10 MIN

Activity 5

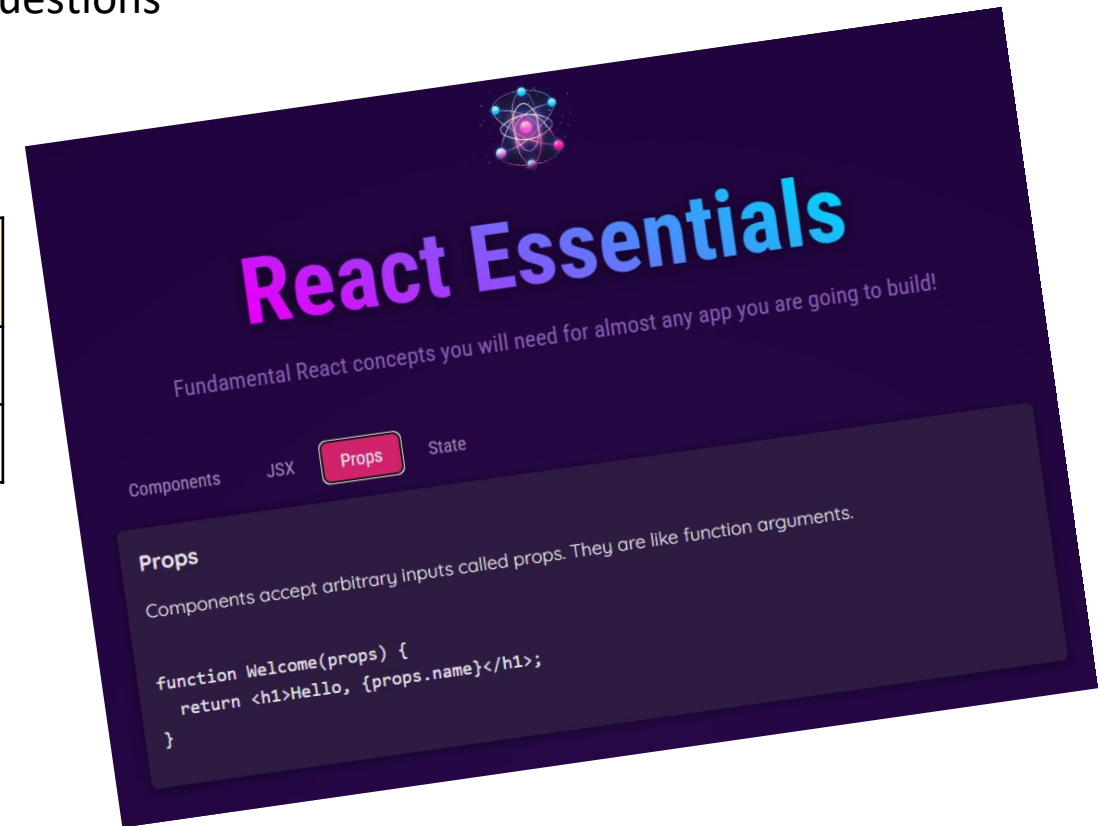
- ✓ **Unzip, install and run** the app
- ✓ In this activity you need to analyze the code and answer to the questions

Q1: **Complete** the state table

Component	State Name	Type	Role	Used for...	Changed when...

Q2: **Draw** the component diagram

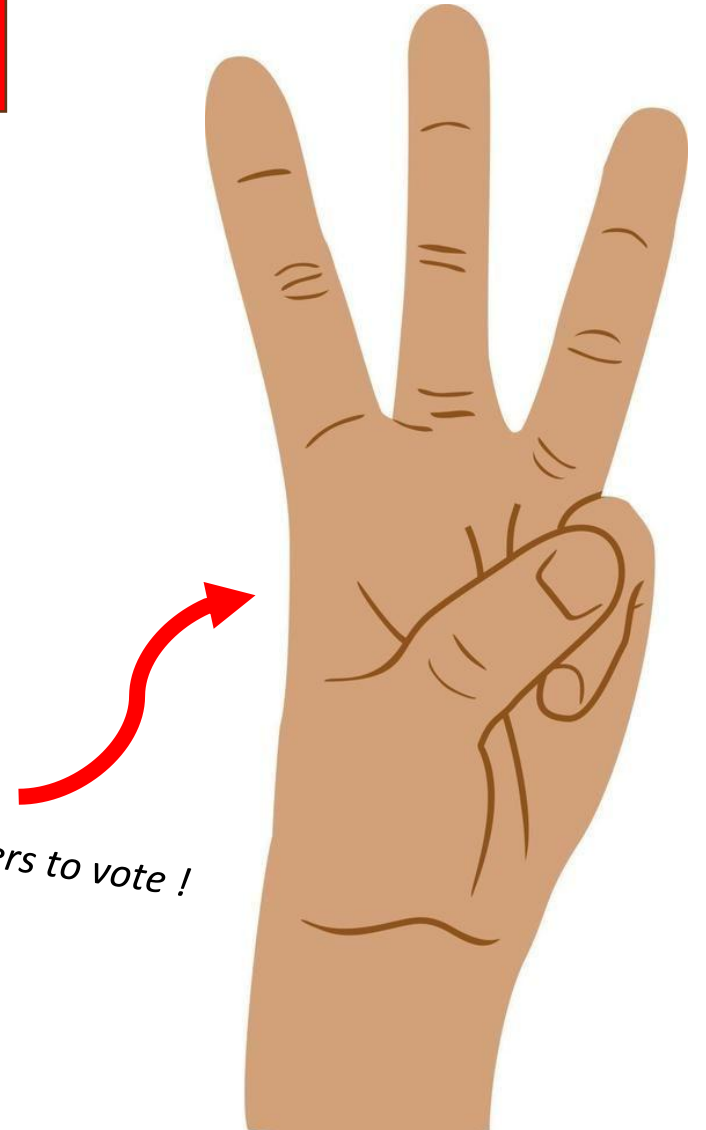
Q3: **Explain** step by step how the content of a tab is displayed when we click on the tab button



Time For

QUIZ !

Use your fingers to vote !



Which of the following code WILL NOT output content conditionally?

1

```

1  function SomeComponent() {
2    const [contentIsVisible, setContentIsVisible] = useState(false);
3
4    function handleClick() {
5      setContentIsVisible(true);
6    }
7
8    return (
9      <div>
10       {contentIsVisible && <p>Visible</p>}
11       <button onClick={handleClick}>Show Content</button>
12     </div>
13   );
14 }

```

2

```

1  function SomeComponent() {
2    const [contentIsVisible, setContentIsVisible] = useState(false);
3
4    function handleClick() {
5      setContentIsVisible(true);
6    }
7
8    return (
9      <div>
10       {contentIsVisible ? <p>Visible</p> : null}
11       <button onClick={handleClick}>Show Content</button>
12     </div>
13   );
14 }

```

3

```

function SomeComponent() {
  const [contentIsVisible, setContentIsVisible] = useState(false);

  function handleClick() {
    setContentIsVisible(true);
  }

  let content;

  if (contentIsVisible) {
    content = <p>Visible</p>;
  }

  return (
    <div>
      {content}
      <button onClick={handleClick}>Show Content</button>
    </div>
  );
}

```

4

```

function SomeComponent() {
  const [contentIsVisible, setContentIsVisible] = useState(false);

  function handleClick() {
    setContentIsVisible(true);
  }

  return (
    <div>
      <p show={contentIsVisible}>Visible</p>
      <button onClick={handleClick}>Show Content</button>
    </div>
  );
}

```

Which of the following code WILL NOT output content conditionally?

1

```

1  function SomeComponent() {
2    const [contentIsVisible, setContentIsVisible] = useState(false);
3
4    function handleClick() {
5      setContentIsVisible(true);
6    }
7
8    return (
9      <div>
10       {contentIsVisible && <p>Visible</p>}
11       <button onClick={handleClick}>Show Content</button>
12     </div>
13   );
14 }

```

2

```

1  function SomeComponent() {
2    const [contentIsVisible, setContentIsVisible] = useState(false);
3
4    function handleClick() {
5      setContentIsVisible(true);
6    }
7
8    return (
9      <div>
10       {contentIsVisible ? <p>Visible</p> : null}
11       <button onClick={handleClick}>Show Content</button>
12     </div>
13   );
14 }

```

3

```

function SomeComponent() {
  const [contentIsVisible, setContentIsVisible] = useState(false);

  function handleClick() {
    setContentIsVisible(true);
  }

  let content;

  if (contentIsVisible) {
    content = <p>Visible</p>;
  }

  return (
    <div>
      {content}
      <button onClick={handleClick}>Show Content</button>
    </div>
  );
}

```

4

```

function SomeComponent() {
  const [contentIsVisible, setContentIsVisible] = useState(false);

  function handleClick() {
    setContentIsVisible(true);
  }

  return (
    <div>
      <p show={contentIsVisible}>Visible</p>
      <button onClick={handleClick}>Show Content</button>
    </div>
  );
}

```

show is NOT an attribute to display/hide an element

Which code WILL WORK and output a list dynamically?

1

```
1 <ul>
2   {[1, 2, 3].transform(number => <li key={number}>{number}</li>)}
3 </ul>
```

2

```
1 <ul>
2   {[1, 2, 3].map(number => <li key={number}>{number}</li>)}
3 </ul>
```

3

```
1 <ul>
2   {[1, 2, 3].list(number => <li key={number}>{number}</li>)}
3 </ul>
```

4

```
1 <ul>
2   {
3     for (const number of [1, 2, 3]) {
4       <li key={number}>{number}</li>
5     }
6   }
7 </ul>
```

Which code WILL WORK and output a list dynamically?

1

```
1 <ul>
2   {[1, 2, 3].transform(number => <li key={number}>{number}</li>)}
3 </ul>
```

2

```
1 <ul>
2   {[1, 2, 3].map(number => <li key={number}>{number}</li>)}
3 </ul>
```

3

```
1 <ul>
2   {[1, 2, 3].list(number => <li key={number}>{number}</li>)}
3 </ul>
```

4

```
1 <ul>
2   {
3     for (const number of [1, 2, 3]) {
4       <li key={number}>{number}</li>
5     }
6   }
7 </ul>
```

What's the problem with this code?

```
1 | <div>
2 |   {someArray.map(item => <p>{item}</p>)}
3 | </div>
```

1

☐ You can only map to `<li>`, not to `<p>`

2

☐ You can only output list data between `<ul>` tags, not `<div>`

3

☐ You should add a `key` prop to the list item

4

☐ You should add a `list` prop to the `<ul>`

What's the problem with this code?

```
1 | <div>  
2 |   {someArray.map(item => <p>{item}</p>)}  
3 | </div>
```

1

☐ You can only map to `<li>`, not to `<p>`

2

☐ You can only output list data between `<ul>` tags, not `<div>`

3

☒ You should add a `key` prop to the list item

4

☐ You should add a `list` prop to the `<ul>`

Go **further** after the class...

- ✓ Read Conditional Display
- ✓ Read A Component's Memory
- ✓ Read Pass function as props
- ✓ Read React Virtual DOM

- ✓ Review these slides
- ✓ Review correction code
- ✓ Update **your notes** on this course

